Adam Flores

Los Angeles · adam@deepsteppe.com · 949.677.9177 · github.com/lebronsunkle

I'm a product engineer at eWorldWideWeb, where I build the tools the company runs on, and an IT and AI student at the University of the Cumberlands (graduating June 2027). I also cofounded YOON Fest, a 1,000 person community electronic music festival, and run Deep Steppe, a growing house music vinyl label. I'm looking for a product engineering or full stack role now, while I finish my degree: taking a real need from a real client to working software, and owning what happens after it ships. What I bring is judgment about what to build, what broke, and whether the fix is the right one. Below are write-ups of the projects I'd most like to talk about in an interview.

1. Domain auction sniper
2. Nameway.com
3. YOON Fest
4. The Deep Steppe label manager agents
5. Auto DM, an Instagram comment to DM bot

Domain auction sniper

Built for a domain investor at eWorldWideWeb · May 2026 to now · in daily use, spending real money · code on GitHub

It has bought domains fully hands off while the client slept, including names like neosono.com and shackmax.com that it held through a dead auction and grabbed the instant they relisted at $50. He's a 28 year veteran of GoDaddy's expiring list with 20,000 domains, not technical, and he now runs his sourcing through this tool instead of the 45 minutes a morning he used to spend scrolling by hand.

Each day it indexes about 935,000 live listings, scores them against what he actually buys, and shows him 30,000 ranked names with GoDaddy and Estibot valuations side by side. He sets a max bid and walks away: the bid fires 1.5 seconds before close. The bidding strategy is his, encoded as one pure function we rewrote together after he watched the first version fire live and changed his mind. Its one hard rule: never place the first bid on a no bid auction, because that attracts competition. Let it die, then buy the closeout. If he gets outbid he gets one text. If he's already bidding on a name himself, the tool stands down rather than bid against him.

It spends real money with no sandbox

GoDaddy's test environment doesn't work for these APIs, so every call is production. Every purchase passes a per purchase cap, a daily cap, a sanity check against the reference price, a kill switch, double buy locks, and a single checked path for dollar conversion, where a slipped decimal turns $2,600 into $26,000. Over the cap, the dashboard asks him to confirm the exact amount instead of silently refusing.

What I got wrong, and what it taught me

The guards never failed. What failed was observability: a real money system can look perfectly healthy while doing nothing. Twice this cost an auction. Once the worker hung for 19 hours with no errors, so I built a heartbeat, a watchdog, and a hard deadline on every cycle. The harder one: GoDaddy had been rate limiting us for weeks, the ban text came back unparseable, the parser called it "false," and the worker read false as "nothing for sale" and waited while the client bought the domain by hand in front of it. I rebuilt the state model so "unknown" can never be mistaken for a fact, added a circuit breaker that goes silent long enough for the ban to expire, wired a second price source so one blind sensor can't blind the system, and made the dashboard turn red. Then I wrote a script that rebuilds any domain's full timeline from the database, because by the time you ask "why didn't it buy," the logs are gone.

Python, FastAPI, async SQLAlchemy and PostgreSQL on Neon, Docker on Fly.io, React on Cloudflare Pages. GoDaddy REST and SOAP, Estibot, Twilio, Resend, GitHub Actions. About 21,000 lines, 250+ commits, 287 tests; the money path can't deploy without them passing. The public repo has credentials, client data and scoring tuning removed.

Nameway.com

eWorldWideWeb · 2026 · nameway.com

Nameway is the storefront for a portfolio of more than 20,000 domains. The old site was hard to shop, its inventory drifted out of sync with the registrar, and there was no way to pay online. I rebuilt it in WordPress, then kept building, because a store this size only works if it runs itself. The first Stripe sale came in September 2026.

What's running today

- A custom theme with a search grid, A to Z and by-length browsing, industry pages, and a landing page for every domain, so each name can rank in search on its own.
- A checkout plugin: Stripe (card and ACH) under $5,000, an Escrow.com flow above it. Webhook signature verification, the async events ACH settlement needs, confirmation emails from a real mailbox instead of "WordPress", and listings that take themselves down when they sell.
- An hourly sync against the GoDaddy account: sold names come off the site, new ones stage as drafts, and a dry-run report prints first so nothing changes by surprise.
- Industry classification built on the 45 USPTO trademark classes, installed as a real taxonomy with buyer-friendly names and applied in reviewed batches with a human check, never one unchecked pass.

The logo pipeline: 19,460 vector logos

Every listing needed artwork, so I built a pipeline around an AI design agent I direct. It generates true vector SVG wordmarks: 19,460 of them at about 38 MB total, where raster images would have run past a gigabyte. Letters are outline paths drawn from the domain string, so a misspelled logo is mechanically impossible. A server-side toolchain imports each file, attaches it as the listing's featured image, crops the viewBox, and caches per-logo geometry so letters render at equal height across 19,000 cards regardless of font or icon.

The part I would defend hardest in an interview is the verification discipline. The generator grades its own work, and I never let that be the only grade. My independent audits caught 4,803 bad word splits on the first pass and 255 more on the second, and the fix that stuck replaced a dictionary with frequency scoring against the catalog's own vocabulary: "heat" appears 38 times in the corpus and "theat" once, so sportheat splits after sport. Names that failed review shipped on a holdback list instead of blocking the other 98 percent. And when the client later changed the entire visual direction, the same wave pipeline repainted all 19,000 logos in an afternoon.

What broke, and what it taught me

- Logos imported by a root process came out root-owned. Apache refused to serve them, a redirect plugin bounced every image URL to the homepage, and the symptom read as broken images when the disease was file permissions. Ownership repair is now a step inside the pipeline, not a cleanup after it.
- The site dropped into maintenance mode at 14:30 two days in a row. The same minute twice is a schedule, not a coincidence: a cron-driven auto update kept dying mid-flight and leaving its lock file behind. I disabled auto updates and made updating a deliberate, backup-first task.
- My CLI tooling fataled twice for different reasons: a plugin's filesystem check pushes WordPress into FTP mode when run as root, and a core function my script relied on had been removed in a newer WordPress. Same lesson both times: plain prepared queries and an explicit filesystem method beat convenient wrappers that change underneath you.
- WordPress files each upload under its parent post's date, so 500 logos scattered across month folders going back to 2020 and my first maintenance script only swept the current month. My globs now match what the system actually does, not what I assumed it does.

Hardening, and where it's going

I also cleaned up the server itself: deleted a web-readable backup file that had been exposing payment gateway credentials, had the key rotated, forced every SVG upload through a sanitizer, and retired stale themes and dependencies.

The next step is convergence. Import, classify, generate, and attach already exist as separate proven tools, and they are becoming one intake command: a new price sheet turns into listings, categories, and logos with a review report, instead of a day of manual work. The

human stays where judgment matters, approving low-confidence classifications and new artwork, and the machine does the rest.

WordPress, PHP, MySQL, WP-CLI, Stripe, Escrow.com, GoDaddy API, bash and Python tooling, WHM and cPanel on Liquid Web.

YOON Fest

Cofounder · @yoonfest

YOON is an independent electronic music festival I cofounded. The first one drew about 150 people. It's now past 1,000 guests, and the 8th edition runs three nights in May 2027 at Sequoia Highland Camp near Kings Canyon.

My side of it:

- Booking the lineup: getting avails from agents, sending offers and negotiating deals with local and international artists.
- Venue deals and permits, including the cost analysis and counter offers.
- Sponsorships, which I lead.
- Ticketing on Shotgun, where I'm working out a multi year partnership, and the marketing that sells the tickets.
- This year's partnership with RAW CUTS, a big house and techno YouTube channel that will film the sets and help with booking and promotion.
- The bars: I hire and schedule the bar staff and put together the drink menu.
- Production and operations on the show days.

The festival's Instagram has about 13,600 followers and 1.1 million video views. Running YOON is where I learned to negotiate and sell, and it's the reason Auto DM exists.

Label manager agents and the Deep Steppe Desk

Deep Steppe · 2026

Deep Steppe is my house label. Every release so far has sold out, but each one comes with a long list of chores: distributor uploads, playlist and radio pitching, socials, press emails and shipping, on top of signing artists and manufacturing. Running it alone, things slipped.

I set up a small team of Claude agents to take on that work. A label manager agent keeps the release calendar and hands tasks to two others: one builds releases with our distributor, the other handles pitching, submission forms, socials and press outreach. None of them can pay for, send or submit anything that needs my approval without it. Everything they do shows up on one dashboard, the Desk, where each release, task and outreach contact links straight to the page or form it needs and gets checked off as it's done.

What the Desk covers:

- Releases: every release on the calendar with its date, its distributor status and links to the store and streaming pages, so I can see what's left before it ships.
- Needs Adam: the short list of things only I can do, like a login, a purchase or a signature, each one linking to the exact page.
- The outreach pipeline: every playlist curator, radio station and press contact, where they stand from drafted to sent to replied, and a link to the outlet or form. For one single, the agents sent individual pitches to more than 40 curators and filled out the free submission forms on their own.
- Drafts waiting on me: pitch emails sit on the Desk with Read draft and Approve buttons. Nothing goes out until I approve it, and once I do, it goes to the whole list.
- Deadlines and socials: pitch windows, submission deadlines and the posting schedule, plus scheduled check-ins, like pulling streaming numbers a few days after a release and pitching another round if they're low.
- Shipping: for our CD release, the agents made a shipping label for every order, cropped them for the label printer, and marked each Bandcamp order shipped with its tracking number in the buyer email.
- Message the team: a chat panel on the Desk where I give the agents instructions from my phone, and they pick them up in their next session.

The Desk updates itself. When an agent finishes something, it checks it off without me asking. And whenever I have to step in, the agent writes what it learned into a shared skill file, so the next run handles that case without me. That's how a curator email that once sat unsent through a release week became a rule: drafts go up for approval the same day they're written.

Claude agents with skills and scheduled tasks, working through Gmail, Google Calendar and Chrome. The Desk is a hosted page with a small shared database the agents read and write.

My first version ran on a different agent platform and didn't hold up, so I rebuilt it on Claude.

Auto DM

For Deep Steppe and YOON Fest · 2026

The label and the festival sell records and tickets on Instagram. Comment to DM tools like ManyChat work, but they cost money every month and can't tell you which post actually sold the tickets. So I wrote my own.

Someone comments on a post and gets a DM with the link. Each post can be set up for a release, for tickets, for a guestlist (it asks for their name by DM, caps the list and keeps a waitlist, and prints a door list), or for a sold out show, where it collects emails for the next batch. Every link in a DM is a tracked short link, and a sync with Shotgun, our ticketing

platform, counts each ticket bought through a DM against the post that sent it.

Instagram webhooks deliver comments instantly once Meta approves the app; until then, and as a backup, it checks for new comments every minute. The same Worker also serves the deepsteppe.com homepage and its mailing list signup.

JavaScript on Cloudflare Workers with a D1 (SQLite) database, the Instagram Graph API with OAuth, webhooks and cron, and the Shotgun Tickets API. About 1,900 lines. Costs nothing to run.

Thanks for reading. The best way to reach me is adam@deepsteppe.com.